What we built

DrugMirror is a drug safety demo: enter a molecule (SMILES), get ML predictions for side effects, binding risk, FAERS signals, and drug–drug interactions. The UI is static HTML; the brain is a Python API with four trained models.

Backend stack

Layer	Technology
API framework	FastAPI (Python)
Server	Uvicorn (uvicorn main:app) on **port 8000**
ML	XGBoost models saved as .pkl (loaded with joblib)
Chemistry	RDKit — parse SMILES, Morgan fingerprints, descriptors
Optional	SHAP explanations, Claude API for AI summaries (mock if no API key)

Why FastAPI: quick to build, auto Swagger docs at /docs, easy JSON APIs, good for hackathons.

Run command:

backend/ → python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000

(or double-click start_backend.bat)

The 4 models (real predictions)

All live under models/ and load at server startup.

Model	File	Endpoint	What it predicts
Side effects	side_effect_models_sider.pkl	POST /predict	Probability per MedDRA-style label (GI, cardiac, etc.)

Model	File	Endpoint	What it predicts
Binding affinity	binding_affinity.pkl	POST /predict (in binding_risk)	pKd / IC50 & risk for **hERG**, **CYP3A4**, **COX-2**
FAERS	faers_models (1).pkl	POST /faers	Real-world adverse-event signals (with PRR cache)
DDI	ddi_model_final.pkl	POST /ddi	Drug–drug interaction severity between two SMILES

Flow for one drug:

1. Frontend sends SMILES as JSON.
2. Backend validates SMILES (RDKit).
3. RDKit builds a 1024-bit Morgan fingerprint + descriptors.
4. Correct model runs predict / predict_proba.
5. JSON response goes back to the UI.

Real vs mock: GET /health returns "mode": "real" and "model_loaded": true when all four pickles loaded. If files are missing, backend falls back to deterministic mock scores (clearly labeled "mode": "mock").

Main API endpoints

Endpoint	Method	Purpose
/health	GET	Backend alive? Models loaded?
/predict	POST	Side effects + binding + molecular props
/faers	POST	FAERS signal analysis

Endpoint	Method	Purpose
/ddi	POST	Interaction between two drugs (two SMILES)
/molecule	GET	2D structure image (base64 PNG)
/docs	GET	Swagger UI for demos / judges

Example request:

POST http://127.0.0.1:8000/predict

{ "smiles": "CC(=O)Oc1ccccc1C(=O)O" }

Example response fields: predictions, binding_risk, molecular_weight, logp, qed, mode: "real".

CORS: configured so the browser on http://localhost:3000 can call the API (required when not opening HTML as file://).

Frontend stack

Layer	Technology
UI	Plain **HTML + CSS + JavaScript** (Google Stitch export)
Styling	Tailwind CSS (CDN)
Server	Python http.server on **port 3000**
API calls	fetch() to http://127.0.0.1:8000

No React, no Vite — just static pages + fetch. Simple for a hackathon demo.

Run command:

frontend/ → python -m http.server 3000

(or start_frontend.bat)

Open: http://localhost:3000 (not the .html file directly).

Frontend ↔ backend mapping

Page	User input	Backend call	What UI shows
index.html	—	GET /health	Green “Backend Online” dot
side-effects.html	Drug / SMILES	POST /predict + GET /molecule	Risk bars, structure image
binding-affinity.html	SMILES	POST /predict	hERG / CYP3A4 / COX-2 panel
real-world-data.html	SMILES	POST /predict + POST /faers	FAERS signals
ddi-checker.html	Two SMILES	POST /ddi	Interaction risk, similarity
analyzer.html	SMILES	POST /predict	Combined analysis view

Typical JS pattern:

```
const API_BASE = 'http://127.0.0.1:8000';

const res = await fetch(`${API_BASE}/predict`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ smiles: userInput })
});

const data = await res.json();

// render data.predictions, data.binding_risk, etc.
```

Loading states, error banners (“Cannot reach backend”), and risk badges (LOW / MODERATE / HIGH) are handled in the page scripts.

End-to-end flow (one sentence for judges)

"User enters a SMILES string in the browser → frontend POSTs to our FastAPI backend → RDKit featurizes the molecule → four XGBoost models score side effects, binding, FAERS, and DDI → results render as risk charts and warnings in the UI."

Folder layout (mental model)

```
DrugMirror/
├── backend/
│   ├── main.py        # FastAPI routes
│   ├── predict.py     # Model loading + inference
│   ├── ddi.py         # DDI logic
│   ├── features.py    # SMILES validation / features
│   └── .venv/         # Python dependencies
├── models/            # 4 trained .pkl files
├── frontend/
│   ├── index.html
│   ├── side-effects.html
│   ├── binding-affinity.html
│   ├── ddi-checker.html
│   └── real-world-data.html
├── start_backend.bat
└── start_frontend.bat
```

Demo checklist (30 seconds before presenting)

1. Run start_backend.bat → wait for "Application startup complete".
2. Run start_frontend.bat.
3. Open http://localhost:3000 → confirm **Backend Online**.

4. Try Aspirin SMILES: CC(=O)Oc1ccccc1C(=O)O on Side Effects page.
5. Optional: show http://127.0.0.1:8000/docs for API credibility.

One-liner pitch

DrugMirror connects a Stitch-designed web UI to a FastAPI + XGBoost backend that turns molecular structure (SMILES) into interpretable safety predictions — side effects, protein binding, real-world FAERS signals, and drug–drug interactions — for research and hackathon demos.